

Task 1: (Divide & Conquer)

- (a) Name the three phases of a *Divide & Conquer* Algorithm and briefly describe each.

	name	description
1.		
2.		
3.		

- (b) The MERGESORT algorithm takes as input an array ($A[1..n]$) and sorts it. In a first step, the algorithm splits the array into two arrays ($A[1.. \lfloor \frac{n}{2} \rfloor]$) and ($A[\lfloor \frac{n}{2} \rfloor + 1..n]$). It then recursively sorts the two arrays before combining them into a single sorted array via the MERGE subroutine, which runs in time $O(n)$. Give the recurrence for the runtime of the algorithm.

$T[n] \leq$

--

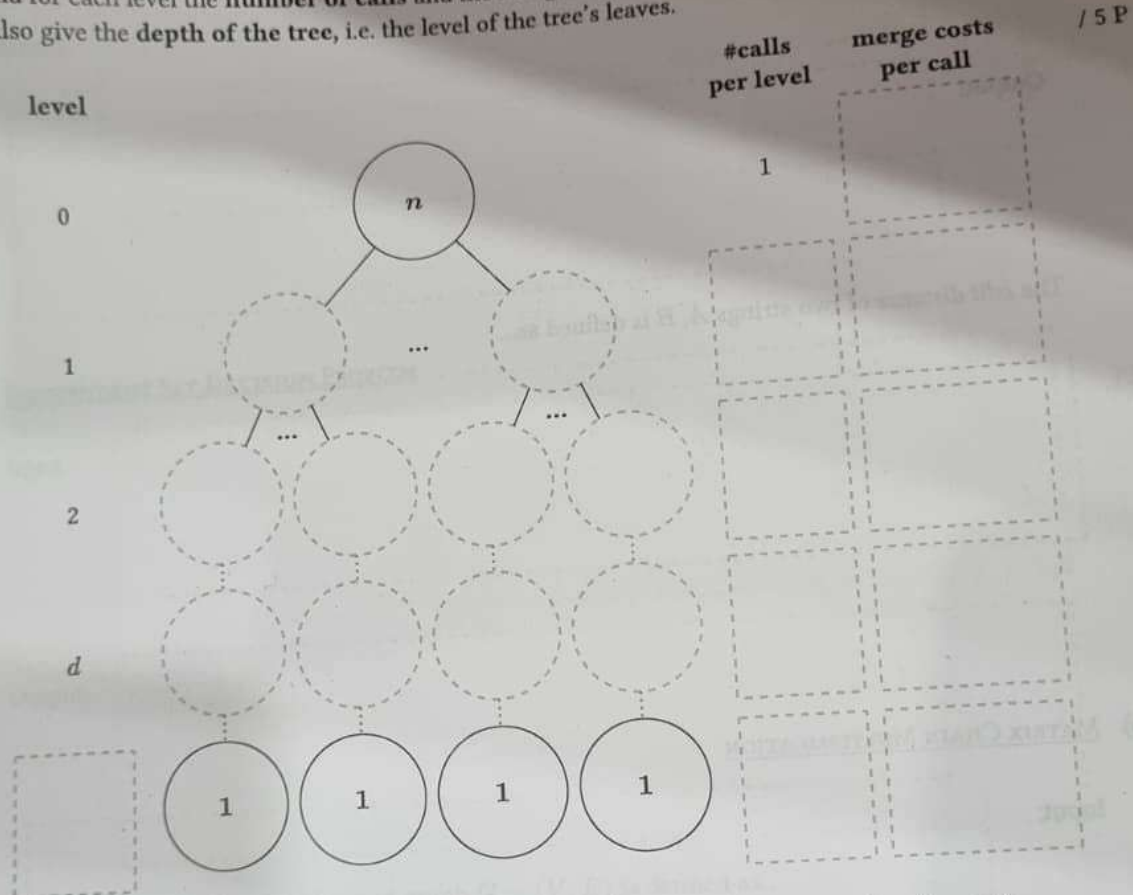
/ 1 P

Task 2: (Recursion Trees)

Let $c > 0$ be an integer number. Let T be the following recurrence that describes for all $n = 3^x$ with $x \in \mathbb{N}$ the runtime of a Divide & Conquer-Algorithm $\mathcal{A}$:

$$T[1] = c \quad \text{and} \quad T[n] = 5 \cdot T\left[\frac{n}{3}\right] + c \log n.$$

- (a) Complete the following recursion tree for T by writing in the instance sizes, and for each level the **number of calls** and the **merge costs per call**. Also give the **depth of the tree**, i.e. the level of the tree's leaves.



- (b) Derive a *non-recursive formula* for the runtime $T[n]$ from the recursion tree, for all $n \geq 2$. (You don't need to simplify the formula.)

$T[n] =$

/ 2 P

- (c) State the pseudocode for the **QUICKSORT**(a, b) algorithm, where the parameters a, b are indices in a global array $A[1 \dots n]$. (Your code may call the **PARTITION** subroutine from the lecture; its pseudocode is printed below.)

QUICKSORT(a, b)

PARTITION(a, b, p)

```

1 swap  $A[p] \leftrightarrow A[b]$ 
2  $\ell \leftarrow a$ 
3 for  $i \leftarrow a, \dots, (b-1)$  do:
4   if  $A[i] < A[b]$  then:
5     swap  $A[\ell] \leftrightarrow A[i]$ 
6      $\ell \leftarrow \ell + 1$ 
7 swap  $A[b] \leftrightarrow A[\ell]$ 
8 return  $\ell$ 

```

Task 3: (Problem Descriptions)

Complete the following descriptions of various computational problems by filling in the blanks.

(a) EDIT DISTANCE

Input:

Output:

The *edit distance* of two strings A, B is defined as...

(b) MATRIX CHAIN MULTIPLICATION

Input:

Output:

(c) TRAVELING SALESMAN PROBLEM

Input:

Output:

/ 2 P

(d) INDEPENDENT SET DECISION PROBLEM

Input:

Output:

/ 3 P

An *independent set* in an undirected graph $G = (V, E)$ is defined as...

Task 4: (Maximum Independent Set on Trees)

Let T be a tree with root vertex r .

As in the lecture, we denote the subtree rooted in a vertex v by T_v , and the children of v by $N(v)$.

We want to compute the size of a **largest independent set** in T .

In the lecture, we solved the problem with the Dynamic Programming algorithm `TreeMaxIS`.

Step 1: Formalization + recursive solution:

- (a) Describe in words the meaning of the values $\text{maxIS}(v)$ for a vertex v in the tree.
(How did we define these values in the formalization step?)

$\text{maxIS}(v)$ denotes

/ 1 P

- (b) Suppose we have computed the value $\text{maxIS}(v)$ for all vertices v .
How can we use this information to find the size of the largest independent set of T ?

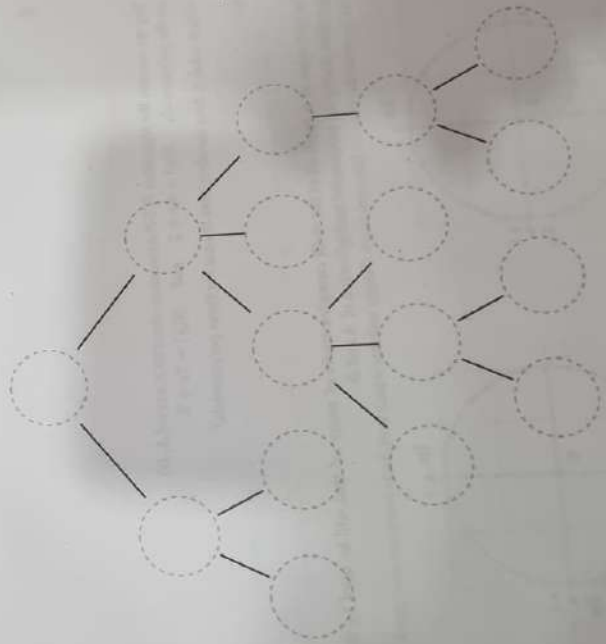
/ 1 P

- 13P

$$\text{MAXIS}(v) =$$

169

- (d) Execute the algorithm that was presented in the lecture on the tree given below by writing the computed values for $\text{MAXIS}(v)$ into the dotted circle at each vertex.



Task 5: (Fast Polynomial Multiplication and FFT)

Let $A(x)$ and $B(x)$ be two polynomials and let $C(x)$ be the polynomial $C = A \cdot B$

of degree $\deg(C) \leq n-1$, for some power of two $n = 2^d$.

In the lecture you have seen the *fast polynomial multiplication algorithm FFT-Polynomial-Multiplication*(A, B) which takes as input two polynomials $A(x), B(x)$ in coefficient representation, and computes as output the coefficient representation of $C(x)$.

- (a) Briefly describe the **three phases** of the algorithm $\text{FFT-Polynomial-Multiplication}(A, B)$.

For each phase, give the final runtime and name any important subroutines.

1.

/ 6 P

2.

3.

- (b) Say we execute the algorithm $\text{FFT-Polynomial-Multiplication}(A, B)$

on the polynomials $A(x) = 4x + 3$ and $B(x) = 7x + 9$.

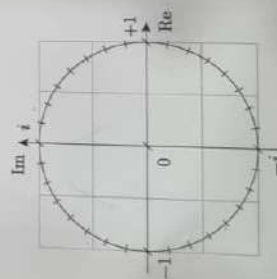
Which value n does the algorithm choose for these polynomials?

 $n =$

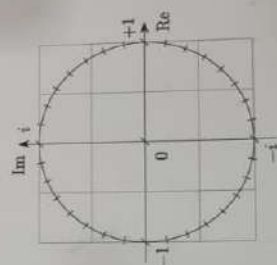
/ 2 P

For this value of n , mark all of the n -th complex roots of unity ω^n that will be used by the algorithm during the polynomial multiplication of A and B .

(Use the second unit circle for corrections. Clearly indicate which unit circle you want us to grade.)



/ 2 P



- (c) Show that the product of all the n -th complex roots of unity is $+1$ if n is odd and -1 if n is even. That is, prove that

$$\prod_{k=0}^{n-1} \omega^k = \begin{cases} +1, & \text{if } n \text{ is odd,} \\ -1, & \text{if } n \text{ is even.} \end{cases}$$

Hint: You may use the fact that for any complex number z we have that $z^a = (z^b)^{a/b}$ where a and b are integers.

Proof:

/ 7 P

$$\prod_{k=0}^{n-1} \omega^k = \omega^0 \cdot \omega^1 \cdot \omega^2 \cdot \dots \cdot \omega^{n-1}$$

Task 6: (Huffman Algorithm)

The following table gives the frequencies of the letters $A, \dots, G$ in an unknown language. (The entries have already been sorted according to their frequencies.)

(a) Run the *Huffman Algorithm* on this input by filling in the table:

$\text{pred}[i]$ denotes the parent ("predecessor") of node i in the resulting coding tree.

For $\text{mark}[i] = 0$, the node i should be the left child of its parent node; for $\text{mark}[i] = 1$, the node i should be the right child.

a_i	G	F	B	A	C	E	D						
i	1	2	3	4	5	6	7	8	9	10	11	12	13
p_i	0.03	0.04	0.05	0.06	0.10	0.29	0.43						
pred													
mark													

(b) Draw the resulting coding tree T !

(Draw the child with edge label 0 as the left child of its parent.)

(c) Compute the cost of the computed Huffman code!

$$\text{Cost}(T, p) =$$

Task 2: (Minimum Spanning Trees - Kruskal's Algorithm)

In this task, $G = (V, E, c)$ is a weighted undirected graph with $n = |V|$ many nodes, $m = |E|$ many edges, and edge weight function $c: E \rightarrow \mathbb{R}$. You can also assume that G is connected.

- (a) A **spanning tree** of G is defined as an edge set $T \subseteq E$, such that...

/ 10 P

/ 2 P

What is the definition of the **cost** of a spanning tree T ?

$\text{Cost}(T) :=$

- (b) Recall that we called a set $R \subseteq E$ **extendible** if there exists an MST T , such that $R \subseteq T$.
Complete the formulation of the **Cut Property**:

/ 4 P

“Assume that the following three conditions hold:

1. $R \subseteq E$ is and
2. $(S, V - S)$ is such that

and

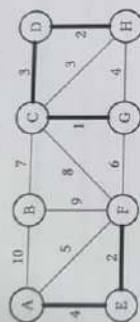
3. $e = \{v, w\}$ is an edge with v and w that

Then,

- (c) Below is a partial execution of Kruskal's algorithm on a graph G :

The graph G and the set R .

Edges that have already been added to the set R are marked in bold.



$R = \{(A, B), \{E, F\}, \{C, G\}, \{C, D\}, \{D, H\}\}$

The associated
Union-Find data structure
at that time of the execution.



- (i) What is the next edge e added to R by Kruskal's algorithm?

$e =$

- (ii) After e is added to R , the Union-Find data structure has to be updated.
Draw the updated Union-Find data structure!

Task 8: (Dijkstra's Algorithm)

- (a) Let $G = (V, E, c)$ be a weighted digraph with $n = |V|$ and $m = |E|$.
State the running time of Dijkstra's algorithm (using a binary heap as priority queue):

$O(\dots)$

- (b) Run Dijkstra's algorithm on the weighted digraph below, starting at vertex S .

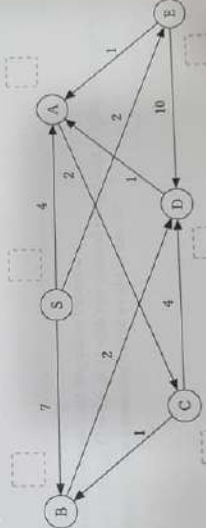
Fill in the fields and tables as follows (see example on the right):

- If node X is processed in round i , write i into the dotted field next to X .
- If node X is found or its distance value $\text{dist}[X]$ is changed in round i , add a new column to X 's table and record
 - the round i ,
 - the new distance $\text{dist}[X]$, and
 - the corresponding (shortest-path tree) predecessor $\text{pred}[X]$.

(X) 13

round	0	2	5	11
dist	∞	24	12	10
pred	—	F	G	H

round	0	round	0	round	0
dist	∞	dist	0	dist	∞
pred	—	pred	—	pred	—



round	0	round	0	round	0
dist	∞	dist	∞	dist	∞
pred	—	pred	—	pred	—

- (c) Suppose we now run Dijkstra's algorithm on a different digraph G starting from a vertex s .
At the end of the algorithm, there is a vertex t such that $\text{dist}[t] = \infty$.
What do we now know about s and t ?

19P

Algorithm (Traveling Salesman Problem)

- 1 Choose start vertex $v_1 \in V$

```

2 for  $v \in V \setminus \{v_1\}$  do:

```

$$3 \quad | P[\emptyset, w] \leftarrow c(v_1, w)$$
4 for $k = 1, \dots, n - 2$ do:

```

5   for  $S \subseteq V \setminus \{v_1\}$  and  $|S| = k$  do:
6     for  $w \in S$  do:

```

```

6   for  $w \notin S \cup \{v_1\}$  do:
7       if  $D[w] < D[v_1] + c(v_1, w)$  then

```

```

7   | | |  $P[S, w] \leftarrow \min_{u \in S} P[S \setminus \{u\}, u] + c(u, w)$ 
8 return  $\min_{u \in V \setminus \{s\}} P[V \setminus \{s\}, u]$ 

```

8 return $\min_{w \in V \setminus \{v_1\}} P[V \setminus \{v_1, w\}, w] + c(w, v_1)$

- / 3 P

Task 3: Divide & Conquer

(a) Name the three phases of a Divide & Conquer Algorithm and briefly describe each.

name	description
1.	
2.	
3.	

- (b) The MergeSort algorithm takes as input an array $A[1..n]$ and sorts it. In a first step, the algorithm splits the array into two arrays $A[1.. \lfloor \frac{n}{2} \rfloor]$ and $A[\lfloor \frac{n}{2} \rfloor + 1..n]$. It then recursively sorts the two arrays before combining them into a single sorted array via the Merge subroutine, which runs in time $O(n)$.

Give the recurrence for the runtime of the algorithm.

$$T(n) \leq$$

/ 1 P

- (b) Prove that the following problems are in the class NP!
- (i) **VERTEX COVER DECISION PROBLEM**
- Input: An undirected graph $G = (V, E)$, an integer k .
 - Output: Is there a set $C \subseteq V$ of size $|C| \leq k$, such that every edge in E is incident to some $v \in C$?

/ 6 P

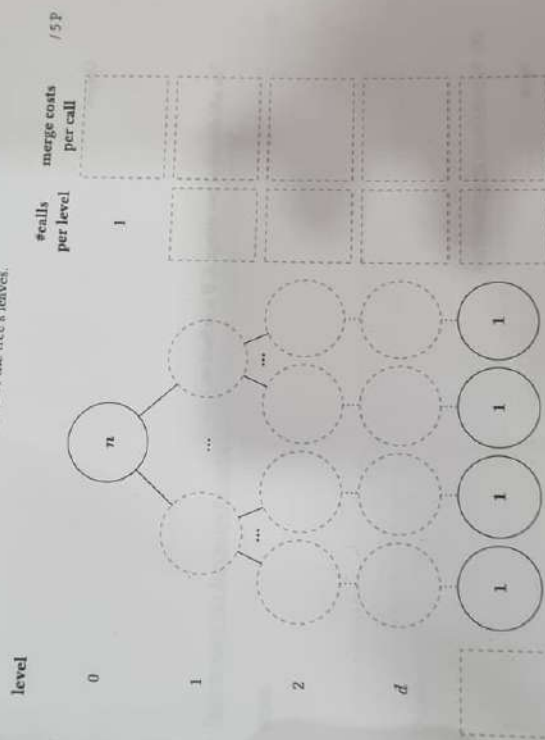
- (ii) **MINIMAL SPANNING TREE DECISION PROBLEM**
- Input: A graph $G = (V, E, c)$ with non-negative integer edge weights, an integer k .
 - Output: Is there a spanning tree of weight at most k ?

Task 2: (Recursion Trees)

Let $c > 0$ be an integer number. Let T be the following recurrence that describes for all $n = 3^x$ with $x \in \mathbb{N}$ the runtime of a Divide & Conquer-Algorithm A .

$$T[1] = c \quad \text{and} \quad T[n] = 5 \cdot T\left[\frac{n}{3}\right] + c \log n.$$

- (a) Complete the following recursion tree for T by writing in the instance sizes, and for each level the **number of calls** and the **merge costs per call**. Also give the **depth of the tree**, i.e. the level of the tree's leaves.



- (b) Derive a *non-recursive formula* for the runtime $T[n]$ from the recursion tree, for all $n \geq 2$. (You don't need to simplify the formula.)

/ 2 P

$T[n] =$

- (c) State the pseudocode for the `QUICKSORT( $a, b$ )` algorithm, where the parameters a, b are indices in a global array $A[1..n]$. (Your code may call the `PARTITION` subroutine from the lecture; its pseudocode is printed below.)

/ 4 P

`QUICKSORT( $a, b$ )``PARTITION( $a, b, p$ )`

```
1 swap  $A[p] \leftrightarrow A[b]$ 
2  $\ell \leftarrow a$ 
3 for  $i \leftarrow a, \dots, (b-1)$  do:
4   if  $A[i] < A[b]$  then:
5     swap  $A[i] \leftrightarrow A[\ell]$ 
6      $\ell \leftarrow \ell + 1$ 
7 swap  $A[b] \leftrightarrow A[\ell]$ 
8 return  $\ell$ 
```

Task 6: (Huffman Algorithm)

The following table gives the frequencies of the letters A, ..., G in an unknown language.
(The entries have already been sorted according to their frequencies.)

- (a) Run the *Huffman Algorithm* on this input by filling in the table!
 $\text{pred}[i]$ denotes the parent ("predecessor") of node i in the resulting coding tree.
 For $\text{mark}[i] = 0$, the node i should be the left child of its parent node; for $\text{mark}[i] = 1$, the node i should be the right child.

/ 4 P

a_i	G	F	B	A	C	E	D						
i	1	2	3	4	5	6	7	8	9	10	11	12	13
p_i	0.03	0.04	0.05	0.06	0.10	0.29	0.43						
pred													
mark													

- (b) Draw the resulting coding tree T !
 (Draw the child with edge label 0 as the left child of its parent.)

/ 4 P